EXPERIMENT NO. 7

SMART CITY TRAFFIC MANAGEMENT SYSTEM USING DYNAMIC DENSITY-BASED TRAFFIC LIGHT CONTROL

ABISHEK DEVRAJ (192512126)

AIM

To design and simulate a smart city traffic management system using Python in Google Colab that dynamically controls traffic signals based on vehicle density and gives priority to the lane having the highest traffic congestion.

OBJECTIVES

1. To simulate a smart city traffic intersection.
2. To generate real-time traffic density for multiple lanes.
3. To identify the lane with maximum traffic density.
4. To dynamically assign the green signal to the busiest lane.
5. To simulate vehicle clearance during each traffic cycle.
6. To reduce traffic congestion using density-based traffic control.
7. To visualize traffic density using graphs.
8. To analyse traffic conditions over multiple cycles.
9. To display the current traffic signal status.
10. To generate a final traffic analysis report.

SOFTWARE / APPARATUS REQUIRED

- Google Colab
- Python 3.x
- Random library
- Matplotlib library
- IPython Display
- Time library

THEORY

A **Smart City Traffic Management System** uses intelligent technologies to monitor traffic conditions and control traffic signals efficiently. Traditional traffic signals generally operate using fixed timings, regardless of the number of vehicles waiting on each road.

A **Dynamic Density-Based Traffic Light Controller** improves traffic management by continuously monitoring the number of vehicles in each lane. The lane having the highest traffic density is identified and given the green signal, while the remaining lanes are assigned red signals.

In this experiment, a four-way traffic intersection consisting of **North, South, East, and West** lanes is simulated using Python. Since physical traffic sensors are not used, vehicle arrivals are randomly generated.

For every traffic cycle:

1. New vehicles arrive at each lane.
2. The current queue of vehicles is calculated.
3. The lane with the highest density is identified.
4. The identified lane receives the GREEN signal.
5. A number of vehicles are cleared from that lane.
6. The remaining traffic is displayed.
7. A graph is generated to visualize traffic density.
8. The process is repeated for the next cycle.

Basic Data Flow

Traffic Lanes

↓

Vehicle Arrival

↓

Traffic Density Calculation

↓

Find Highest Density

↓

Select Busiest Lane

↓

GREEN Signal

↓

Clear Vehicles

↓

Update Traffic Queue

↓

Display Graph

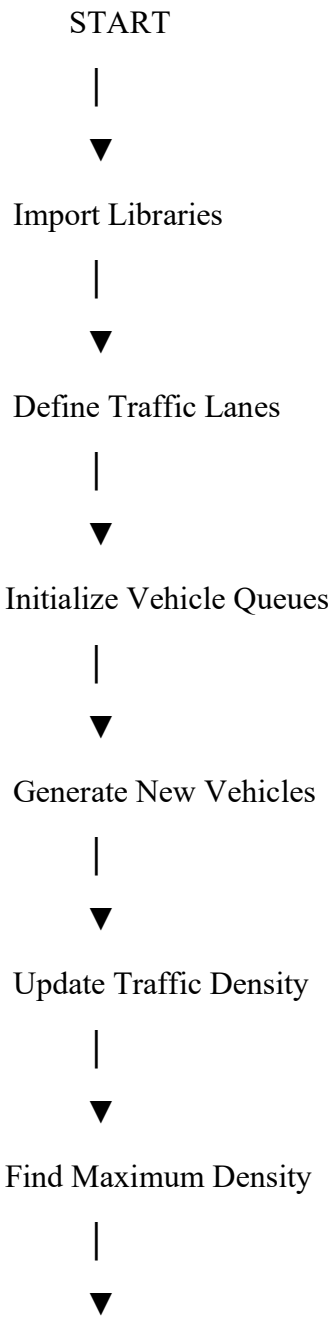
↓

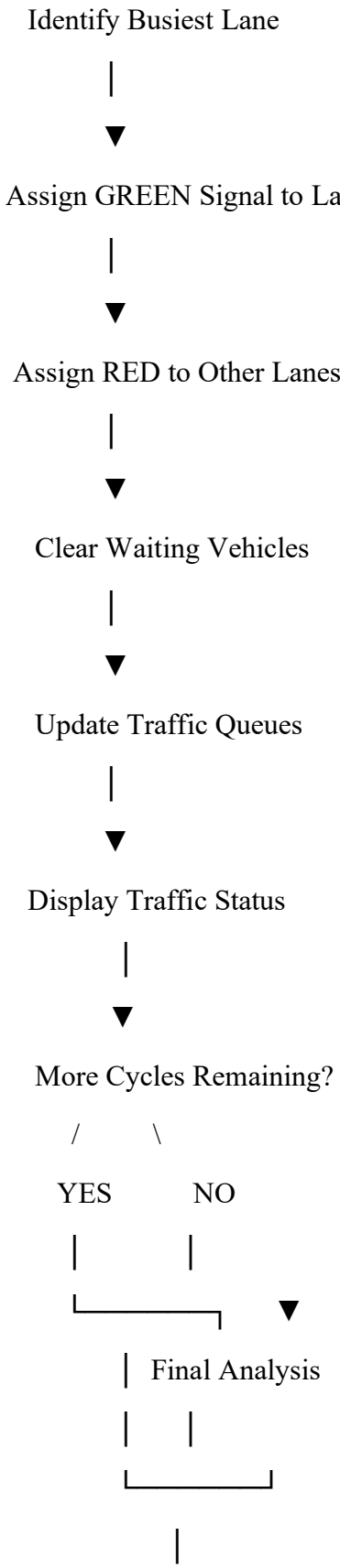
Next Cycle

ALGORITHM

1. Start the program.
2. Import the required Python libraries.
3. Define the four traffic lanes: North, South, East, and West.
4. Initialize the number of vehicles in each lane.
5. Set the total number of simulation cycles.
6. Generate new vehicles randomly for all lanes.
7. Add the newly arrived vehicles to the existing queues.
8. Compare the vehicle density of all lanes.
9. Identify the lane having the maximum number of vehicles.
10. Assign the GREEN signal to the busiest lane.
11. Assign RED signals to the remaining three lanes.
12. Generate the vehicle clearance capacity.
13. Remove the cleared vehicles from the active lane.
14. Update the traffic statistics.
15. Display the current traffic density.
16. Display the traffic signal status.
17. Plot the current traffic density using a bar graph.
18. Wait for the next traffic cycle.
19. Repeat the process for all simulation cycles.
20. Display the total vehicles cleared from each lane.
21. Display green signal allocation for each lane.
22. Display the remaining vehicles.
23. Plot the final traffic density graph.
24. Display the completion message.
25. Stop the program.

FLOWCHART


START
Import Libraries
Define Traffic Lanes
Initialize Vehicle Queues
Generate New Vehicles
Update Traffic Density
Find Maximum Density


Identify Busiest Lane
Assign GREEN Signal to Lane
Assign RED to Other Lanes
Clear Waiting Vehicles
Update Traffic Queues
Display Traffic Status
More Cycles Remaining?
YES
NO
Final Analysis

▼

END

GOOGLE COLAB CODE

```
# ============================================================
# EXPERIMENT NO. 7
# SMART CITY TRAFFIC MANAGEMENT SYSTEM
# DYNAMIC DENSITY-BASED TRAFFIC LIGHT CONTROLLER
# GOOGLE COLAB VERSION
# ============================================================

import random
import time
import matplotlib.pyplot as plt
from IPython.display import clear_output



# ============================================================
# 1. TRAFFIC LANE CONFIGURATION
# ============================================================

lanes = ["North", "South", "East", "West"]



# ============================================================
# 2. INITIAL TRAFFIC QUEUES
# Different sample values
# ============================================================

queues = [7, 4, 6, 2]



# ============================================================
# 3. SIMULATION SETTINGS
# ============================================================

simulation_steps = 18
delay = 1.5
```

```python
# ============================================================
# 4. STATISTICS STORAGE
# ============================================================

total_cleared = {
    "North": 0,
    "South": 0,
    "East": 0,
    "West": 0
}

green_count = {
    "North": 0,
    "South": 0,
    "East": 0,
    "West": 0
}

cycle_records = []


# ============================================================
# 5. START MESSAGE
# ============================================================

print("=" * 70)
print("        SMART CITY TRAFFIC MANAGEMENT SYSTEM")
print("=" * 70)
print()
print("Dynamic Density-Based Traffic Light Controller")
print()
print("Number of Traffic Lanes :", len(lanes))
print("Simulation Cycles       :", simulation_steps)
print()

time.sleep(2)


# ============================================================
# 6. MAIN TRAFFIC SIMULATION
# ============================================================
```

```
for cycle in range(1, simulation_steps + 1):

    # ------------------------------------------------------
    # Generate new vehicles arriving at each lane
    # ------------------------------------------------------

    new_cars = [
        random.randint(1, 4)
        for _ in range(4)
    ]



    # ------------------------------------------------------
    # Add new vehicles to the queues
    # ------------------------------------------------------

    for i in range(4):
        queues[i] += new_cars[i]



    # ------------------------------------------------------
    # Find the lane having maximum traffic density
    # ------------------------------------------------------

    active_index = queues.index(max(queues))

    active_lane = lanes[active_index]



    # ------------------------------------------------------
    # Give GREEN signal to busiest lane
    # ------------------------------------------------------

    signal_status = []

    for lane in lanes:

        if lane == active_lane:
            signal_status.append("GREEN")
        else:
            signal_status.append("RED")
```

```
# ----------------------------------------------------------
# Calculate number of vehicles cleared
# ----------------------------------------------------------

clearance = random.randint(4, 8)

cars_cleared = min(
    queues[active_index],
    clearance
)



# ----------------------------------------------------------
# Remove cleared vehicles
# ----------------------------------------------------------

queues[active_index] -= cars_cleared



# ----------------------------------------------------------
# Update statistics
# ----------------------------------------------------------

total_cleared[active_lane] += cars_cleared
green_count[active_lane] += 1



# ----------------------------------------------------------
# Store cycle information
# ----------------------------------------------------------

cycle_records.append({
    "Cycle": cycle,
    "North": queues[0],
    "South": queues[1],
    "East": queues[2],
    "West": queues[3],
    "Green Lane": active_lane,
    "Vehicles Cleared": cars_cleared
})
```

```python
# ============================================================
# CLEAR PREVIOUS OUTPUT
# ============================================================

clear_output(wait=True)


# ============================================================
# DISPLAY CURRENT TRAFFIC INFORMATION
# ============================================================

print("=" * 70)
print("        SMART CITY TRAFFIC MANAGEMENT SYSTEM")
print("=" * 70)

print()
print("Traffic Cycle        :", cycle, "/", simulation_steps)
print("New Vehicles Arrived :", sum(new_cars))
print("Busiest Lane         :", active_lane)
print("Vehicles Cleared     :", cars_cleared)

print()
print("CURRENT TRAFFIC DENSITY")
print("-" * 40)

print("North :", queues[0], "vehicles")
print("South :", queues[1], "vehicles")
print("East  :", queues[2], "vehicles")
print("West  :", queues[3], "vehicles")


# ============================================================
# TRAFFIC SIGNAL STATUS
# ============================================================

print()
print("TRAFFIC SIGNAL STATUS")
print("-" * 40)

print("1. North →", signal_status[0])
```

```
print("2. South →", signal_status[1])
print("3. East  →", signal_status[2])
print("4. West  →", signal_status[3])


# ============================================================
# ACTIVE SIGNAL INFORMATION
# ============================================================

print()
print("ACTIVE SIGNAL")
print("-" * 40)

print(
    "GREEN SIGNAL :",
    active_lane
)

print(
    "ACTION       :",
    cars_cleared,
    "vehicles cleared from",
    active_lane
)


# ============================================================
# LIVE TRAFFIC GRAPH
# ============================================================

plt.figure(figsize=(10, 5))

# All lanes RED
colors = [
    "red",
    "red",
    "red",
    "red"
]

# Busiest lane GREEN
colors[active_index] = "green"
```

```
bars = plt.bar(
    lanes,
    queues,
    color=colors
)


# Display vehicle values above bars
for bar, value in zip(bars, queues):

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.5,
        str(value),
        ha="center",
        fontsize=12,
        fontweight="bold"
    )


plt.xlabel("Traffic Lane")
plt.ylabel("Number of Vehicles Waiting")

plt.title(
    "Dynamic Traffic Density - Cycle "
    + str(cycle)
    + " | Green Signal: "
    + active_lane
)


# Set suitable Y-axis
maximum = max(queues)

plt.ylim(
    0,
    max(35, maximum + 8)
)


plt.grid(
    axis="y",
```

```
        linestyle="--",
        alpha=0.5
    )


    plt.tight_layout()

    plt.show()


    # ============================================================
    # WAIT FOR NEXT CYCLE
    # ============================================================

    if cycle < simulation_steps:
        time.sleep(delay)


# ============================================================
# 7. FINAL REPORT
# ============================================================

print()
print("=" * 70)
print("               FINAL TRAFFIC ANALYSIS")
print("=" * 70)

print()

print("Total Simulation Cycles :", simulation_steps)


# ============================================================
# VEHICLES CLEARED
# ============================================================

print()
print("VEHICLES CLEARED BY LANE")
print("-" * 40)

for lane in lanes:
```

```
    print(
        lane,
        ":",
        total_cleared[lane],
        "vehicles"
    )



# ============================================================
# GREEN SIGNAL ALLOCATION
# ============================================================

print()
print("GREEN SIGNAL ALLOCATION")
print("-" * 40)

for lane in lanes:

    print(
        lane,
        ":",
        green_count[lane],
        "cycles"
    )



# ============================================================
# FINAL SIGNAL STATUS
# ============================================================

final_active_index = queues.index(max(queues))
final_active_lane = lanes[final_active_index]

print()
print("FINAL TRAFFIC SIGNAL STATUS")
print("-" * 40)

for lane in lanes:

    if lane == final_active_lane:
        print(
            lane,
            "→ GREEN"
```

```
        )
    else:
        print(
            lane,
            "→ RED"
        )


# ============================================================
# REMAINING VEHICLES
# ============================================================

print()
print("REMAINING VEHICLES")
print("-" * 40)

for i in range(4):

    print(
        lanes[i],
        ":",
        queues[i],
        "vehicles"
    )


# ============================================================
# 8. FINAL TRAFFIC GRAPH
# ============================================================

plt.figure(figsize=(10, 5))


bars = plt.bar(
    lanes,
    queues
)


# Display values
for bar, value in zip(bars, queues):
```

```
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.5,
        str(value),
        ha="center",
        fontsize=12,
        fontweight="bold"
    )


plt.xlabel("Traffic Lane")
plt.ylabel("Vehicles Remaining")

plt.title(
    "Final Traffic Density After Simulation"
)


plt.ylim(
    0,
    max(35, max(queues) + 8)
)


plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.5
)


plt.tight_layout()

plt.show()


# ============================================================
# 9. COMPLETION MESSAGE
# ============================================================

print()
print("=" * 70)
```

```
print("Traffic Simulation Completed Successfully!")
print("Dynamic Traffic Control Applied Successfully!")
print("=" * 70)
```

INPUT

Number of Traffic Lanes = 4

Traffic Lanes:

1. North
2. South
3. East
4. West

Initial Vehicle Queues:

North = 7

South = 4

East = 6

West = 2

Number of Simulation Cycles = 18

New Vehicle Arrival = Random

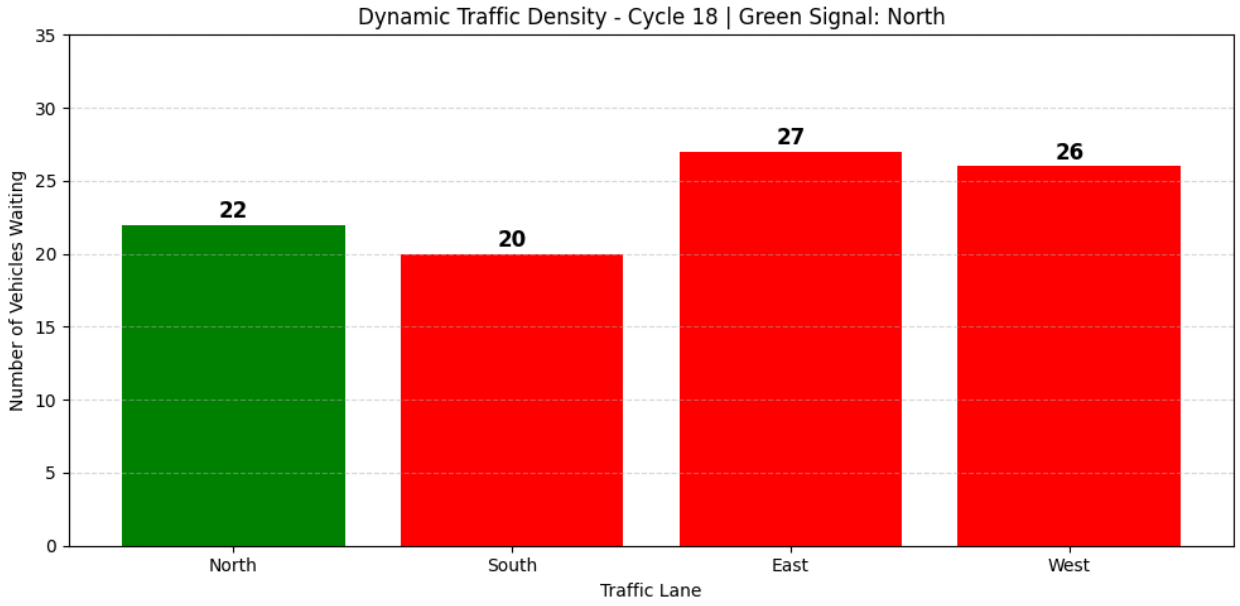
Vehicle Clearance = Dynamically Generated

SAMPLE OUTPUT

Note: The actual values will vary because the vehicle arrivals and vehicle clearance values are randomly generated.

```
================================================================
======
       SMART CITY TRAFFIC MANAGEMENT SYSTEM
================================================================
======

Traffic Cycle        : 8 / 18
New Vehicles Arrived : 12
Busiest Lane         : East
Vehicles Cleared     : 7

CURRENT TRAFFIC DENSITY
---------------------------------------
North : 18 vehicles
South : 14 vehicles
East  : 21 vehicles
West  : 16 vehicles

TRAFFIC SIGNAL STATUS
---------------------------------------
1. North → RED
2. South → RED
3. East  → GREEN
4. West  → RED

ACTIVE SIGNAL
---------------------------------------
GREEN SIGNAL : East
ACTION       : 7 vehicles cleared from East
```

OUTPUT



RESULT

The **Smart City Traffic Management System using Dynamic Density-Based Traffic Light Control** was successfully designed and simulated using Python in Google Colab.